

Hoopla

Full Redesign Reference — the system, the inventory, and the rules.

Everything in the app and how it actually works, read from the shipping codebase. Prepared for the product designer and the engineering team ahead of a full redesign.

1.3.11

BUILD (30)

30

SCREENS

62

LAYOUTS

24

ADAPTERS

13

VIEWMODELS

3

LOCALES

0

CART

`uz.alphazet.hoopla` — Coffee loyalty & ordering, Uzbekistan.

Android only · minSdk 26 · XML + ViewBinding · Material 2 · Russian default, Uzbek, English · Dark mode ships.
13 July 2026.

Part 1 — The System

1.1 Architecture

```
Fragment/Activity → ViewModel → Repository → Retrofit Service → api.hoopla.uz/api/
```

- **Three modules:** `:app` (UI, DI) · `:domain` (base classes, repos, network, cache) · `:data` (services, DTOs)
- **State:** every async result is a `UIResource<T>` — `Loading` / `Success<T>` / `Error`. Every screen is a `when` over those three, so **every screen already has three states by construction — the redesign must draw all three.**
- **DI:** Koin · **Nav:** Cicerone + AndroidX Navigation · **Images:** Coil · **Maps:** Google Maps + clustering · **Paging:** Paging 3

1.2 The interceptor chain — this is UX, not plumbing

```
Chucker (debug) → NetworkConnection → NetworkInterceptor (412→401) → Logging → TokenAuthenticator
```

`TokenAuthenticator` silently refreshes the token on a 401. A **repeated** 401 logs the user out — so a customer can be ejected mid-flow. The redesign needs a graceful re-auth path, not a hard bounce to login.

1.3 HTTP status codes are the navigation logic

The most under-documented part of the app, and it is pure business logic.

CODE	MEANING	WHAT THE APP DOES
401	Not authenticated	→ <code>AuthActivity</code> , after a silent refresh attempt
402	Payment required	Two contradictory behaviours. From Checkout → opens <code>checkout_url</code> in the external browser and closes the order as created. From OrderActivity → opens SubscriptionActivity .
428	Precondition required	→ <code>PaymentServicesActivity</code> . In practice: <i>insufficient balance, go top up.</i>
412	—	Rewritten to 401 by the interceptor
400 · 403 · 404 · 409 · 422 · 429 · 5xx · offline	Typed exceptions	Surfaced through <code>RemoteErrorListener</code> on the base Activity / Fragment

The 402 split is almost certainly a bug: the same server signal means “buy a subscription” on one screen and “pay for your coffee” on the next.

Part 2 — What's in the App

2.1 Navigation shell

Bottom navigation, four tabs: Home · Map · Orders · Profile.

The “Map” title is a **hardcoded English literal**, not a string resource — untranslated in a Russian-default app.

Guest mode exists. Profile carries `authGroup` / `unAuthGroup`; unauthenticated users browse freely and hit login walls at Order and at Checkout — two different depths.

2.2 Every screen — 30 total

ENTRY

SCREEN	CONTAINS
Splash	Logo, intro container
Onboarding	Image, title, description, page indicators, Skip, Next
Auth — phone	Phone input, Send
Auth — OTP	Code input, phone echo, error text, resend with countdown timer , privacy-policy link

HOME TAB

Header (logo, search, **notification bell with unread badge**), then a collapsing app bar containing:

- **Stories rail** — seen / unseen ring
- **Active orders rail** — live orders in progress
- **Categories rail** — drink-category filter
- **Nearby partners list** — distance-sorted
- **GPS-off card** — prompts to enable location
- **QR-scan floating button**
- Swipe-to-refresh
- **Loyalty stamp card — present in the layout, commented out in code**

DISCOVER

SCREEN	CONTAINS
Map	Google Map, marker clustering, custom markers, logo overlay
Search	Search input, partner results list
Shop detail	Collapsing header image, shop name, working-hours row , paused badge , Call, Directions, category chip group , drinks by category, working-time table, image carousel, contact-URL list

ORDERING

SCREEN	CONTAINS
Order	Drink image, name, café, modifier groups (dynamic) <i>or</i> legacy Size / Sugar / Milk / Syrup sections, comment field , live-updating price button
Checkout	Drink summary, modifier line items, promocode row (add / applied / remove), cashback card (toggle, amount picker, change), discount rows, earn-cashback preview , total, Order button
Cashback picker (sheet)	Balance, +/- in 1 000 steps with long-press repeat, Use, Reset
Promocode input (sheet)	Code field, validate

ORDERS TAB

- **Subscription QR card** — brand name, tariff name, phone, QR
- **Daily-limit card** — used / available / progress bar · **commented out**
- Order history list, empty state, swipe-refresh
- **Order info** — image, name, shop, status, order number, comment, line items, promo discount, cashback used, total, **cashback earned**, **Rate**, **Pickup QR**, **Fiscal receipt**, **Cancel order**, **Continue payment**
- **Order QR** (sheet) — pickup code

PROFILE TAB

- Auth / guest states, name + edit, phone number
- **Balance card + Top up** · **Gift-card card + redeem**
- My details · Subscription · Language · Theme · **Devices** · Support · Privacy Policy · Terms of Use · Settings · Log out / Log in
- **Edit profile** — name, **gender spinner**, **date of birth**, **Delete account**
- **Devices** — active sessions (device, platform, app version, IP, last active), revoke
- **Subscriptions** — plan list with feature bullets; Lottie empty state
- **Payment services** — provider list → top-up sheet
- Sheets: Language · Theme · Redeem gift card · Gift-card success (full-screen) · Input name

CONTENT

SCREEN	CONTAINS
Notifications	List, mark-read
Notification detail	Full message
Story viewer	Full-screen pager, per-slide progress bars, auto-advance, tap / hold, deep-link out
Feedback (sheet)	Post-order rating

2.3 Component inventory

24 adapters — Category, Loyalty, NearShop, Notification, Story, StoryGroupPager, StorySlide, Modification, ModificationType, ModifierGroup, ModifierOption, Size, SummaInfo, ActiveOrder, Order, Device, PaymentService, Feature, Subscription, Partner, Drinks, Images, Url, WorkTime.

13 ViewModels — Auth, Home, Notification, StoryViewer, Order, Orders, Profile, Devices, GiftCard, Payment, Subscription, Search, Shop.

10 dialogs & bottom sheets — feedback, gift-card success, input name, input promocode, order QR, redeem gift card, select cashback amount, select language, select theme, top-up.

Part 3 — The Business Logic

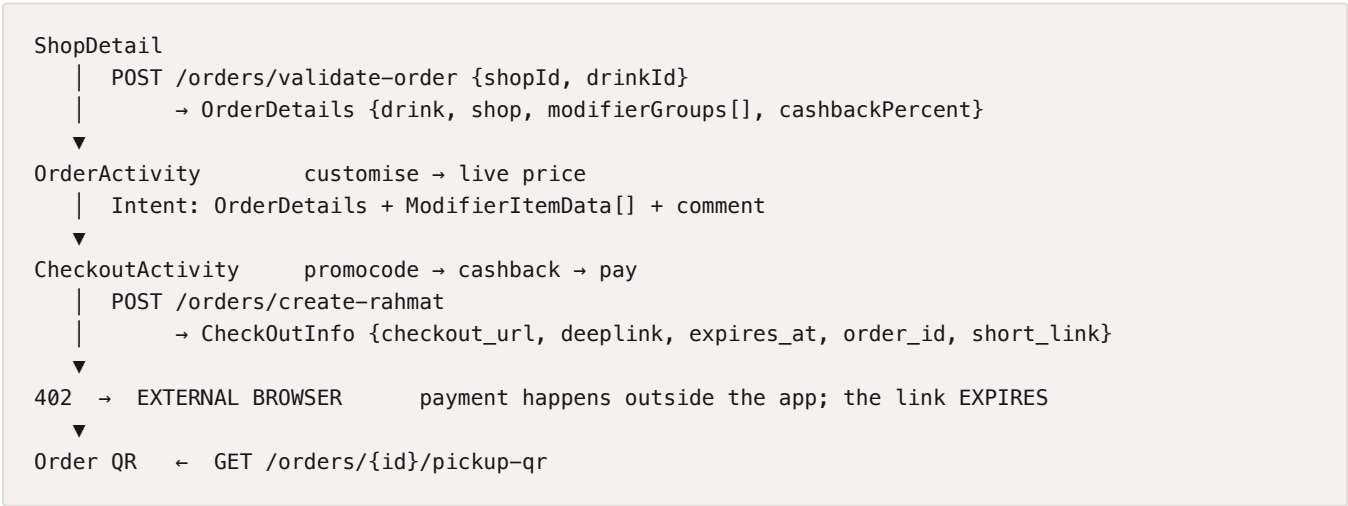
Two findings that change the redesign

- 1. **The loyalty card is switched off.** The feature is built end-to-end — endpoint, model, adapter, layout — and the calls are **commented out** in `HomeScreen.kt:147,150`. So do the subscription's daily-drink allowance calls in `OrdersVM.kt:51`. In a coffee *loyalty* app, the stamp card is dark. The only live retention mechanic is cashback.
- 2. **"Cashback" spends the wallet.** `UserData` has two money fields — `balance` and `cashback_balance`. Every screen that says "cashback" reads `balance`. `cashback_balance` is **never read anywhere in the app**.

3.1 Money model

CONCEPT	FIELD	REALITY
Wallet balance	<code>balance</code>	Topped up by payment services and gift cards. This is what checkout actually spends.
Cashback balance	<code>cashback_balance</code>	Returned by the API. Never read anywhere.
Currency	<code>currency</code>	Returned. Never used — <code>"UZS"</code> is hardcoded onto every price.
Cashback rate	<code>cashback_percent</code>	Per-shop, per-order, from <code>validate-order</code> . Zero ⇒ the earn row hides.

3.2 The ordering funnel



There is no cart. `OrderItemData` is singular and zero cart or basket references exist anywhere in the codebase. A second drink means the whole funnel again plus a second payment — items-per-order is capped at 1.00 by construction.

Payment is not in-app. `create-rahmat` hands off to a browser checkout URL with an expiry. `POST /orders/create` exists in the service, repo and ViewModel — and is **never called**.

3.3 Pricing algebra — exact

```
subtotal    = drink.amount +  $\Sigma$ (selected modifier.modifierPrice)
afterPromo  = max(0, subtotal - promo.discountAmount)      ← promocode applies FIRST
total       = max(0, afterPromo - cashbackUsed)            ← cashback applies SECOND
earned      = total × shop.cashbackPercent / 100           ← earned on what was ACTUALLY PAID
```

Two rules that must not break: **promo before cashback**, both clamped at zero; and **cashback is earned on the post-discount total** — so spending cashback reduces the cashback earned back.

3.4 Cashback spend rules

```
maxSpendable = min(user.balance, subtotal - promo.discountAmount)
```

- Adjusted in **1 000 UZS steps**; long-press auto-repeats every 300 ms
- **Opens pre-filled at the maximum**, not at zero
- Minus snaps a non-round amount down to the nearest 1 000 first
- **Applying a promocode after setting cashback silently trims the cashback to fit** — the customer is not told
- The toggle is disabled entirely when `balance ≤ 0`

3.5 Promocode rules

```
POST /orders/check-promocode {code, shopId, drinkId, modifiers[]} → {valid, code, discountType, discountValue, discountAmount, subtotal, total}
```

- Validated **against the specific drink and its modifiers** — not shop-generic
- The **server** computes `discountAmount`; the app never calculates it
- `discountType` / `discountValue` (percent vs fixed) are returned but **unused in the UI**
- One code at a time; re-sent on `create-rahmat` and re-validated at purchase

3.6 Drink customisation — two systems, side by side

The app runs whichever the API returns.

NEW — `modifierGroups[]` (used when non-empty). Each group carries `key`, `name`, `minSelect`, `maxSelect` (null = unlimited). `isRequired = minSelect ≥ 1`; `isSingleSelect = maxSelect == 1`. The order button is **blocked** until every group is satisfied.

LEGACY — fixed `{size, sugar, milk, syrup}` (fallback). Four hardcoded single-select sections, no validation.

The first option of every legacy section is auto-selected and its price is added to the total. Customers are charged for a size, milk or syrup they never actively chose. This is a live pricing rule, not a UI default — removing it changes what people pay.

3.7 Order lifecycle

```
pending_payment → paid → pending → completed
├── payment_failed    retryable
├── payment_expired   link dead → must create a NEW order
├── cancelled
└── error             café / vendor rejected
```

`pending_payment` awaiting the external payment · `paid` money in, pushed to the café · `pending` café accepted, drink being made · `completed` ready. Cancel via `POST /orders/{id}/cancel`. Active orders surface on Home.

3.8 Subscription

- `GET /subscriptions` → `{id, name, price, days, currency, features[]}` · `POST /subscriptions/buy`
- The active plan hangs off `UserData.subscription {id, name, endDate, endDateUnix}` — **time-boxed, not recurring**
- **Daily allowance:** `GET /orders/drinks-stat` → `{available, used}` — the “N drinks per day” mechanic. **UI commented out.**
- Upsold only from inside `OrderActivity`, on a 402

3.9 Loyalty card — built and switched off

`GET /orders/loyalty-card` → `{drinkIndex, isFilled, isFree}[]` — a classic stamp card: N slots, `isFilled` = earned, `isFree` = the reward slot. Endpoint, model, adapter and layout all exist. **The calls are commented out.**

3.10 Gift cards

`POST /gift-cards/redeem {code}` → `{credited, balance, currency}`. A **one-time top-up straight into the wallet balance** — not a separate purse.

3.11 Auth & sessions

```
phone → POST /auth/login      → {sessionId, sessionExpiresAt} → SMS
      → POST /auth/confirm-sms {code, sessionId, deviceName, platform, deviceId, appVersion}
                                      → {userId, isNewUser, jwt{accessToken, refreshToken, expireAt}}
```

- `isNewUser` drives the name-capture prompt
- Sessions are **device-bound** — `GET /user/devices`, `DELETE /user/devices/{id}` to revoke
- Refresh: `PATCH /user/refresh-token?refreshToken=` — as a **query parameter** (already flagged to move into the body)
- `DELETE /user/deactivate` — account deletion exists

3.12 Shops

`ShopItemData {shopId, name, pictureUrl, distance, location, logoUrl, acceptingOrders, pausedUntil}`

A café can be PAUSED (`acceptingOrders: false` + `pausedUntil`). Badge and scrim drawables already exist. The shop list, the map and the menu must all handle it. Nearby: `GET /shops/near-shops` (lat/lng). Menu: `GET /shops/drinks` → categories → drinks.

3.13 Stories

`StorySlideData {imageUrl, title, description, duration, linkType, linkValue}` where `linkType ∈ {partner, drink, url}`. Slides auto-advance on `duration`; each deep-links to a partner, a drink, or an external URL. `isSeen` drives the unread ring.

3.14 QR codes — two of them, both expiring

- `GET /user/generate-qr-code` → **identity QR** `{token, expiresAt}` — who you are
- `GET /orders/{id}/pickup-qr` → **pickup QR** `{token, expiresAt, orderId}` — this specific drink

Both expire. Neither screen currently shows a countdown or refreshes.

3.15 Feedback & notifications

`GET /orders/feedbacks/pending` returns **one** order awaiting a rating → `POST /orders/feedbacks/{id}/feedback`. Notifications: list, detail, mark-read; `UserData.unreadNotifications` drives the Home badge.

Part 4 — Dead, Disabled, Contradictory

#	THING	STATUS
1	Loyalty stamp card	Fully built — commented out
2	Subscription daily-drinks allowance	Fully built — commented out
3	<code>cashback_balance</code>	Returned by the API, never read — the app spends <code>balance</code> and calls it cashback
4	<code>currency</code> field	Returned, never used — “UZS” hardcoded
5	<code>POST /orders/create</code>	In service, repo and ViewModel — never called
6	<code>OrderActivity2</code> + <code>screen_order2.xml</code>	An abandoned redesign — never launched
7	402 handling	Two contradictory flows from the same signal
8	Legacy modifier auto-select	Charges for options never chosen
9	<code>discountType</code> / <code>discountValue</code>	Returned, unused
10	“Map” tab title	Hardcoded English literal in a Russian-default app
11	Three strings	Missing from <code>en</code> and <code>uz</code> — 181 ru / 178 / 178

Part 5 — Design System State

- **The semantic colour layer is dead.** `surface`, `onSurface`, `background`, `onBackground`, `error`, `outline` and ten more are **every one of them** `#66000000` — a placeholder 40 % black. Nothing consumes them; screens reach straight into raw ramps. There is, functionally, no theme.
- **The brand ramp is the wrong brand.** `primary` is burgundy `#862631`, but `primary_50...primary_500` is a **teal** ramp (`#0D6A75`, `#3DBCCA`) with a single maroon spliced in at `primary_300`. Leftover from another product. Treat the teal as debris, not as an option.
- **No type scale.** 18 distinct sizes, 10 → 40 sp, across roughly 170 hardcoded declarations. Inter in three weights, plus a display face (Eugusto) used five times.
- **No spacing scale.** 31 distinct dp values across 485 declarations — though 16, 12, 8 and 20 already dominate, so a 4-pt grid is most of the way there by accident. Cheap to fix.
- **Dark mode works by inverting the ramps** in the night resources, so `black_500` renders as near-white `#FAFAFA`. It ships and cannot regress — but **never trust a colour name in this codebase**.
- **Material 2**, no dynamic colour. **117 icons** across two modules sharing no grid, stroke weight, or naming convention.

Part 6 — Decisions Before Design Starts

#	QUESTION	OWNER	WHY IT BLOCKS
1	Is single-item ordering a backend constraint or a product choice?	Backend	Decides whether the cart is a design job or an engineering project. Note: <code>OrderInfo.items[]</code> is already an <i>array</i> of <code>{item_type, name, price}</code> — the backend may already model line items. Worth ten minutes to check.
2	Is <code>balance</code> the same as <code>cashback_balance</code> ?	Backend	Every number on Checkout depends on the answer
3	Is the loyalty card coming back, or is it dead?	Product	It is the premise of the product
4	Is the subscription allowance being shown?	Product	Nobody can currently see what they bought
5	Which 402 behaviour is correct?	Product	Same signal, two flows
6	Is external browser payment permanent, or is in-app payment the goal?	Product	Decides whether checkout is redesigned or merely handed off
7	Should the legacy auto-selected modifier keep charging?	Product	It is a pricing rule, not a default
8	Is <code>#862631</code> locked, or is a rebrand in scope?	Product	Repair job versus fresh identity

The short version

Hoopla is a one-drink-at-a-time ordering app with an external payment handoff, whose loyalty card and subscription allowance — the two mechanics that justify calling it a loyalty app — are built but commented out, and whose “cashback” actually spends the wallet. The design system underneath has no working theme layer and a palette borrowed from a different product. A full redesign has to answer the eight questions above before it draws a single screen.

Hoopla · `uz.alphazet.hoopla` · build 1.3.11 (30) · 13 July 2026.

Every statement in this document was verified against the shipping codebase. Set in Inter and Eugusto — the app's own typefaces.